ReLU Variants Explained

Leaky ReLU | PReLU | ELU | SELU

➢ Introduction:

ReLU (Rectified Linear Unit) became very popular because it's simple and fast:

$$f(x) = max(0, x)$$

1. Works well for many layers
2. But has a major issue: *Dying ReLU Problem*.

➢ Dying ReLU Problem

What Happens:

When inputs to a ReLU neuron become **negative**, the output is **0**.

If this continues during training (due to large negative weights or high learning rate), the neuron **never activates again** — it's "dead."

$$f'(x) = 0 \text{ for } x < 0$$

Hence, the gradient becomes **0** → weights stop updating → neuron never recovers.

➢ Why It Happens:

- Large negative weights or biases
- High learning rate causing big parameter jumps
- Poor initialization

➢ Solutions to Dying ReLU

To overcome this:

1. Use **ReLU variants** that allow small negative outputs
2. Apply **Batch Normalization**
3. Use **lower learning rates**
4. Try **different weight initialization methods**

Now, let's see the **ReLU variants** that solve this problem.

➢ Leaky ReLU (LReLU)

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

where (\alpha) = small constant (like 0.01)

➡️ Instead of giving 0 for negative inputs, it allows a **small negative slope**.

➢ Advantages

- Solves **Dying ReLU** problem (neurons can recover)
- Keeps small gradient for negative values
- Simple and computationally efficient

➢ Disadvantages

- Small negative values may still reduce representation power
- The constant α must be chosen manually

➢ Parametric ReLU (PReLU)

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{if } x \leq 0 \end{cases}$$

Here, (a) is **learned automatically** (not fixed like in Leaky ReLU).
So, during training, the model decides the best negative slope!

➢ Advantages

- Learns the best slope for each neuron
- More flexible than Leaky ReLU
- Handles Dying ReLU problem automatically

➢ Disadvantages

- Slightly more computation
- Risk of **overfitting** (since more parameters are added)

➢ Exponential Linear Unit (ELU)

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$

where (alpha) ≈ 1
Negative inputs produce smooth, small negative outputs (not hard zero).

➢ Advantages

- Reduces **bias shift** (mean of activations closer to 0)
- Helps **faster convergence**
- Provides smooth gradient flow even for negative inputs

➢ Disadvantages

- More **computationally expensive** (exponential)
- Can still saturate for very negative x (gradient → 0)

➢ Scaled Exponential Linear Unit (SELU)

$$f(x) = \begin{cases} \lambda x & \text{if } x > 0 \\ \lambda\alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$

Typical values: (alpha = 1.6733, lambda = 1.0507)

Designed for **Self-Normalizing Neural Networks (SNNs)**.

➡️ SELU automatically keeps activations at **zero mean and unit variance**, so your network stays stable during training.

➢ Advantages

- Automatically normalizes activations (no BatchNorm needed)
- Very stable for deep networks
- Prevents vanishing/exploding gradients

➢ Disadvantages

- Works best with **LeCun Normal Initialization**
- Only effective when used in **fully connected (dense)** layers (not CNNs)

➢ Comparison Table

Function	Formula (for x < 0)	Learnable?	Smooth?	Fixes Dying ReLU?	Notes
ReLU	0	❌	❌	❌	Simple, fast
Leaky ReLU	αx (α=0.01)	❌	✅	✅	Keeps small negative output
PReLU	a·x	✅	✅	✅	Learns slope during training
ELU	α(e^x−1)	❌	✅	✅	Keeps mean close to 0
SELU	λ·α(e^x−1)	❌	✅	✅	Self-normalizing networks

➢ Outro / Summary

✅ **ReLU** is fast and works well but can “die”

✅ **Leaky ReLU** gives a small slope for negative values

✅ **PReLU** learns that slope automatically

✅ **ELU** and **SELU** use exponentials to smooth negative side

✅ **SELU** keeps activations normalized → faster, stable training

---

➢ Simple Viva Definition

ReLU variants are improved activation functions designed to prevent the “dying neuron” issue by allowing small gradients for negative inputs. Common types include Leaky ReLU, PReLU, ELU, and SELU.
